

МТЭС, МКУТ, Програм хангамж

Ц. Саруулчимэг 23B1NUM1396

Documentation Types Mapping Matrix - US-1.1

Эх сурвалж: Chinchilla, Ch. 2 "Understanding Different Types of Documentation" (pp. 9–24)

1. Stripe (docs.stripe.com)

Getting Started

Энэ хэсэг "Stripe-г апп руугаа хамгийн хурдан хэрхэн холбох вэ?" гэсэн хэрэглэгчийн асуултад хариулдаг. Ердийн байрлал нь docs.stripe.com/get-started дэх Quickstart хуудас юм.

Tutorial

Энэ хэсэг "Захиалгын (subscription) төлбөрийн урсгалыг хэрхэн бүрэн хэрэгжүүлэх вэ?" гэх мэт тодорхой хэрэглээний сценарид алхам алхмаар хариулдаг. Ердийн байрлал нь docs.stripe.com/billing/subscriptions/build-subscriptions зэрэг сэдэв тус бүрийн guide хуудас юм.

Reference

Энэ хэсэг "Checkout Session объектод ямар талбарууд, төрлүүд байдаг вэ?" гэсэн бүтцийн асуултад хариулдаг. Ердийн байрлал нь docs.stripe.com/api/checkout/sessions/object дэх object schema хуудас юм.

API Docs

Энэ хэсэг "POST /v1/charges endpoint-д ямар параметр дамжуулах, хариу нь ямар байх вэ?" гэсэн endpoint-специфик асуултад хариулдаг. Ердийн байрлал нь docs.stripe.com/api дор буй, OpenAPI спецификациас автоматаар үүсгэгдсэн endpoint хуудас юм.

2. FastAPI (fastapi.tiangolo.com)

Getting Started

Энэ хэсэг "FastAPI-г хэрхэн суулгаж, хамгийн энгийн endpoint-г бичих вэ?" гэсэн асуултад хариулдаг. Ердийн байрлал нь fastapi.tiangolo.com/ нүүр хуудасны Installation/First Steps хэсэг юм.

Tutorial

Энэ хэсэг "Path parameter, request body, dependency injection-г бодит жишээгээр хэрхэн

ашиглах вэ?" гэсэн алхам алхмаар суралцах сэдэвт хариулдаг. Ердийн байрлал нь [fastapi.tiangolo.com/tutorial/ dotorg](https://fastapi.tiangolo.com/tutorial/dotorg) дараалсан хуудсууд (Path Params, Body, Dependencies г.м.) юм.

Reference

Энэ хэсэг "Depends() функцийн параметрууд, буцаах утга нь юу вэ?" гэсэн код-түвшний асуултад хариулдаг. Ердийн байрлал нь fastapi.tiangolo.com/reference/ дэх, `mkdocstrings`-ээр `docstring`-с автоматаар үүсгэгдсэн хуудас юм.

API Docs

Энэ хэсэг "Миний бичсэн FastAPI апп өөрөө ямар OpenAPI/Swagger спецификаци үүсгэдэг, үүнийг хэрхэн харах вэ?" гэсэн асуултад хариулдаг. Ердийн байрлал нь автоматаар үүсэх `/docs`, `/redoc` хуудсуудын тухай тайлбарладаг fastapi.tiangolo.com/how-to/extending-openapi/ юм.

3. NumPy (numpy.org)

Getting Started

Энэ хэсэг "NumPy-г хэрхэн суулгаж, анхны `array`-гаа үүсгэх вэ?" гэсэн асуултад хариулдаг. Ердийн байрлал нь numpy.org/doc/stable/user/quickstart.html юм.

Tutorial

Энэ хэсэг "Шугаман алгебр эсвэл зургийн боловсруулалтын бодлогыг NumPy-гаар яаж бодох вэ?" гэсэн бодит хэрэглээний асуултад хариулдаг. Ердийн байрлал нь numpy.org/numpy-tutorials/ дэх жишээ `tutorial` хуудсууд юм.

Reference

Энэ хэсэг "`np.reshape()` функц ямар параметртэй, ямар алдаа шидэж болох вэ?" гэсэн функц/классын дэлгэрэнгүй асуултад хариулдаг. Ердийн байрлал нь numpy.org/doc/stable/reference/generated/numpy.reshape.html дэх `docstring`-с автоматаар үүсгэгдсэн хуудас юм.

API Docs

Энэ хэсэг "NumPy-г C өргөтгөл дотроос хэрхэн дуудах, `PyArray_*` функцүүд юу хийдэг вэ?" гэсэн доод түвшний, хэлээс хамаарсан `interface`-ийн асуултад хариулдаг. Ердийн байрлал нь numpy.org/doc/stable/reference/c-api/index.html юм.

Audience Persona Card - Cluvr баримт бичгийн уншигч

Эх сурвалж: Bhatti et al., *Docs for Developers*, Ch. 1 "Understanding Your Audience" (Persona p.15, User Stories p.16, User Journey Maps p.17)

Persona: Төгөлдөр

Талбар	Утга
Нэр (First Name)	Төгөлдөр
Зохиомол компани (Fictional Company)	CampusLoop Labs: оюутны клубуудад зориулсан жижиг веб-платформууд хөгжүүлдэг хамтарсан багийн зохиомол нэр
Техникийн үүрэг (Technical Role)	Бакалаврын 3-р курсын Junior Frontend Developer, Cluvr-тэй төстэй UI-г бие даан кодлож, багийн бусад гишүүдийн үлдээсэн README/тохиргооны заавраар л ажилладаг
Өдөр тутмын хэрэгсэл (Daily Toolstack)	VS Code, Tailwind CSS + PostCSS/Node build хэрэгсэл, Chrome DevTools, GitHub Desktop, Figma (wireframe-үүдийг лавлах)

3 бодит хөгжүүлэгчийн "rain point" (нийгэмлэгийн бодит эх сурвалжаас)

1. Tailwind-ийн эхлэл тохиргоо хэт хүнд мэт санагддаг

Жижиг, статик HTML төсөлд Node.js суулгаж, config файл эхлүүлж, build процесс тохируулах шаардлагатай болдог нь "Lego байшин барихад кран авчирсантай адил" гэж developer community-д (dev.to) байнга гомдоллодог сэдэв. Cluvr шиг цэвэр HTML/Tailwind/vanilla JS төсөлд энэ friction нэн ялангуяа мэдэгддэг.

2. Mobile hamburger menu-ийн JS toggle логик тогтвортой ажилладаггүй

GitHub болон DEV Community дээрх олон thread-д (жишээ нь "Crafting a mobile menu with Tailwind CSS without JavaScript" dev.to) Tailwind-ийн peer-* эсвэл custom JS toggle-той хослуулахад class нь зарим sibling/nested элемент дээр ажиллахгүй байх зэрэг тогтмол давтагддаг асуудал байдаг. Энэ бол Cluvr-ийн өөрийн хөгжүүлэлтэд тааралдсан hamburger-menu bug-той яг адилхан класс шинж чанартай асуудал.

3. Utility class-уудыг цээжлэх шаардлагатай мэт мэдрэгдэж, docs хайлт удаашрах

Community-ийн feedback-д (Scrimba, DEV Community) шинэхэн хөгжүүлэгчид зуу зуун Tailwind utility class-г цээжлэхийг оролдож цаг үрдэг, харин IntelliSense/docs search ашиглах хэрэгтэйг мэддэггүй тохиолдол их гардаг гэж дурддаг энэ нь "docs-оо хараад

биш, санамжаар код бичих" гэсэн ерөнхий асуудалтай холбоотой (Stack Overflow 2024 Developer Survey-ийн дагуу хөгжүүлэгчдийн 44% нь docs хангалтгүй үед эх кодыг шууд уншиж ойлгохыг оролддог).

Constructed quote (30 минутын ярилцлагаас сэдэвлэсэн)

"Би Figma-гаас mobile wireframe-ийг харж байгаад л desktop дээр яг ямар class ашиглаж layout-оо хийхээ мэдэхгүй удаж байлаа. README дээр зөвхөн npm install гэсэн байсан ч ямар Tailwind тохиргоо хэрэгтэй, hamburger menu яагаад заримдаа нээгддэггүйг тайлбарлаагүй байсан, би зүгээр л ажиллах хүртэл санамсаргүй байдлаар class нэмж, хасаж байсан." Төгөлдөр 30 минутын ярилцлагаас

US-1.3: "Curse of Knowledge" Self-Audit: Cluvr README

Литературын эх сурвалж: Bhatti et al., *Docs for Developers*, Ch. 1 p. 3 "The Curse of Knowledge" (Newton 1990; Camerer et al. 1989) & Ch. 3 p. 58 "State your most important information first"

Тест хийсэн хэрэглэгч (persona): Төгөлдөр Junior Frontend Developer, CampusLoop Labs, Cluvr-ийн UI-г бие даан кодлодог

0. Контекст: Cluvr гэж юу вэ

Cluvr нь оюутны клубуудыг нэгтгэсэн, event/арга хэмжээнд бүртгүүлэх (apply хийх) боломжтой веб-платформ. Одоогийн README нь Vite-ийн стандарт template дээр үндэслэсэн бөгөөд Cluvr-д тусгайлсан мэдээлэл (Tailwind тохиргоо, hamburger menu логик) нэмж бичсэн хувилбар.

1. ЭХ ХУВИЛБАР (BEFORE) Одоогийн README

Cluvr

Cluvr - оюутны клубуудын event/application-уудыг нэгтгэсэн платформ.

Tech Stack

React + Vite ашигласан болно HMR болон ESLint дүрмүүдтэй.

Currently, two official plugins are available:

- [[@vitejs/plugin-react](#)] uses Oxc
- [[@vitejs/plugin-react-swc](#)] uses SWC

Getting Started

```
npm install
npm run dev
```

Tailwind Configuration

Tailwind CSS ашиглахын тулд PostCSS pipeline-г тохируулж, tailwind.config.js дотор content path-уудыг зааж өгөх шаардлагатай.

Mobile Menu

Hamburger menu нь peer-* selector ашиглан toggle хийгддэг тул харгалзах sibling элементийн class order чухал.

Expanding the ESLint configuration

If you are developing a production application, we recommend using TypeScript with type-aware lint rules enabled.

2. AUDIT "Jargon-only / Opaque" гэж тэмдэглэсэн өгүүлбэрүүд

Төгөлдөрийн pain point-той (Tailwind setup хэцүү, hamburger menu bug, docs хайлт удаашрах) шууд харьцуулж 4 өгүүлбэрийг шалгав:

#	Эх өгүүлбэр	Tag	Яагаад Төгөлдөрт ойлгомжгүй вэ
1	"Currently, two official plugins are available: @vitejs/plugin-react uses Oxc / @vitejs/plugin-react-swc uses SWC"	Jargon-only	"Oxc", "SWC" гэдэг нь compiler/tooling нэр боловч тайлбаргүй. Аль нэгийг сонгох шаардлагатай юу, ялгаа нь юу вэ гэдэг эхлэгчид тодорхойгүй. Curse of knowledge: зохиогч эдгээрийг өдөр тутам ашигладаг тул "мэдэгдэхүйц" гэж бодсон.
2	"Tailwind CSS ашиглахын тулд PostCSS pipeline-г тохируулж, tailwind.config.js дотор content path-уудыг зааж өгөх шаардлагатай"	Overly opaque	"PostCSS pipeline", "content path" гэдэг нэр томъёог тайлбарлаагүй, ямар файл руу, ямар утга бичихийг заагаагүй. Энэ бол Төгөлдөрийн #1 pain point ("Lego байшин барихад кран авчирсантай адил") — яг тохиолдож буй friction.

3	<i>"Hamburger menu нь peer- selector ашиглан toggle хийгддэг тул харгалзах sibling элементийн class order чухал"</i> *	Jargon-only	Энэ өгүүлбэр Төгөлдөрийн бодит quote-той шууд холбоотой ("hamburger menu яагаад заримдаа нээгддэггүйг тайлбарлаагүй"). "peer-*", "sibling class order" гэдгийг тайлбарлаагүй бөгөөд хамгийн чухал зүйл — юу хийхийг заагаагүй, зөвхөн яагаад эвдэрдэг тухай tech-хийсвэрлэл өгсөн.
4	<i>"...we recommend using TypeScript with type-aware lint rules enabled"</i>	Overly opaque	"type-aware lint rules" гэдэг нь ESLint-ийн дэвшилтэт ойлголт (typescript-eslint parserOptions.project). Шинэхэн хөгжүүлэгчид энэ нь юу хийдэг, яагаад хэрэгтэйг мэдэхгүй, зөвхөн нэр дурдсан.

3. REWRITE "Most Important Information First" дүрэм (Bhatti p. 58)

Хамгийн их friction үүсгэдэг #3 (hamburger menu) өгүүлбэрийг сонгов, учир нь энэ бол Төгөлдөрийн бодит бичсэн bug-тай шууд таарч байна.

Асуудал (Before)

Өгүүлбэр эхлээд техникийн шалтгааныг (peer-* selector) тайлбарлаад, хэрэглэгчид хамгийн хэрэгтэй мэдээлэл болох **"юу хийхийг"** төгсгөлд нь ч гэсэн огт өгөхгүй орхисон. Уншигч эхлээд яагаад гэдгийг унших ёстой болж, шийдлийг олохоос өмнө математик шиг debug хийх шаардлагатай болно.

Шинэ хувилбар (After) хамгийн чухал мэдээлэл эхэнд

Mobile Menu

Menu нээгдэхгүй бол: HTML дотор toggle-checkbox нь menu-тэй шууд sibling (ижил түвшний, зэргэлдээ) байгаа эсэхийг эхлээд шалгана уу.

Жишээ нь:

ЗӨВ (checkbox ба menu шууд sibling):

```
<input type="checkbox" id="menu-toggle" class="peer hidden" />
<nav class="hidden peer-checked:block">...</nav>
```

БУРУУ (checkbox ба menu хооронд wrapper <div> орсон тул peer-checked ажиллахгүй):

```
<input type="checkbox" id="menu-toggle" class="peer hidden" />
<div>
  <nav class="hidden peer-checked:block">...</nav>
</div>
```

Шалтгаан: Tailwind-ийн `peer-\*` selector нь зөвхөн шууд дараагийн sibling элемент дээр л ажилладаг тул завсарт нэмэлт element орвол menu тогтмол эвдэрдэг.

Юу өөрчлөгдсөн бэ: Actionable зөвлөмж ("шалгах зүйл") хамгийн эхэнд, дараа нь concrete код жишээ (зөв vs буруу), хамгийн сүүлд техникийн шалтгаан. Ингэснээр Төгөлдөр шиг хэрэглэгч эхний мөрийг уншаад л асуудлаа шалгаж эхэлж чадна, техникийн тайлбарыг уншиж дуустал хүлээх шаардлагагүй.

4. BEFORE / AFTER DIFF (Course Workspace-д нийтлэх хувилбар)

Mobile Menu

- Hamburger menu нь peer-* selector ашиглан toggle хийгддэг тул
- харгалзах sibling элементийн class order чухал.

- + Menu нээгдэхгүй бол: HTML дотор toggle-checkbox нь menu-тэй
- + шууд sibling байгаа эсэхийг эхлээд шалгана уу.

+

+ ЗӨВ:

- + `<input type="checkbox" id="menu-toggle" class="peer hidden" />`

- + `<nav class="hidden peer-checked:block">...</nav>`

+

- + БУРУУ (завсарт `<div>` орсон):

- + `<input type="checkbox" id="menu-toggle" class="peer hidden" />`

- + `<div>`

- + `<nav class="hidden peer-checked:block">...</nav>`

- + `</div>`

+

- + Шалтгаан: `peer-\*` selector зөвхөн шууд дараагийн sibling дээр

- + л ажилладаг.

Шалгуур	Before	After
Хамгийн чухал мэдээлэл байрлал	Төгсгөлд ч алга (шалтгаан л бий)	Эхний мөрөнд юу шалгахыг шууд хэлнэ
Concrete жишээ	Алга	код блок хоёулаа бий
Jargon тайлбарлагдсан эсэх	Үгүй (peer-*, sibling)	Тийм жишээгээр харуулсан

Actionable эсэх	Үгүй, зөвхөн тайлбар	Тийм, шууд алхам өгсөн
-----------------	----------------------	------------------------

5. Дүгнэлт

Curse of Knowledge-ийн шинжлэх ухааны үндэслэл (Newton 1990 fingers-tapping experiment: tapper 51% таамаглаж байхад listener 2.5% л таамагласан) энд ч харагдаж байна зохиогч (Cluvr багийн туршлагатай гишүүн) peer-* selector, PostCSS pipeline, Охс/SWC зэрэг нэр томьёог "ойлгомжтой" гэж бодсон боловч Төгөлдөр шиг шинэхэн хэрэглэгчид эдгээр нь бүрэн jargon байсан. README-г MIF (Most Important Information First) дүрмээр дахин бичих нь уншигчид actionable мэдээллийг эхэнд нь өгч, техникийн шалтгааныг сонголтоор дараа нь уншихаар үлдээдэг.

US-1.4: "Writer-in-the-Middle" Team Value Proposition

- **Асуулт:** *What value can I deliver as a documentation author in a software team?*
- **Контекст:** Cluvr оюутны клубуудын event/apply платформ (CampusLoop Labs)
- **Литературын эх сурвалж:** Chinchilla, Ch. 1, pp. 1–8 (Why care about tech writing? p.1; What can documentarians accomplish? pp.2–3; Writer in the middle, p.5; End users vs. end-end users, p.7)

200-Word Response (Sprint Planning Advocacy)

As Cluvr's documentation author, my value extends far beyond writing a README, I act as the "writer-in-the-middle" (Chinchilla, p. 5), sitting between engineering, product, marketing, sales, support, and DevRel to close information gaps and break down content silos across the team.

For Marketing, SEO-optimized copy on our public clubs-directory page drives organic search traffic from students discovering new clubs. For Product, a clear apply-flow quick-start builds user confidence in Cluvr's onboarding. For Sales, a feature doc aligned with CampusLoop's pitch deck gives club leads a ready-made demo script. For Support, an FAQ addressing "Apply button does nothing" reduced repeat tickets measurably. For DevRel, an API quick-start lets partner clubs integrate their own event feeds. For Engineering, a debug guide for the hamburger-menu bug saves developer time, proving that "documentation makes engineering look good" (p. 3). For Machine Readers, structured markdown gives LLMs and chatbots an accurate source for answering questions about Cluvr.

Finally, my persona work extends past direct users like junior developers to "end-end users" (p. 7) students who never touch the platform but experience its output at events. Their experience is shaped entirely by documentation quality, which is why I advocate writing time during sprint planning.

Acceptance Criteria *GIVEN-WHEN-THEN* Format

Scenario 1: 7 Spheres of Impact

- **GIVEN** I am drafting a 200-word value-proposition response for sprint planning
- **WHEN** I describe my impact as a documentation author
- **THEN** I detail all 7 spheres (Marketing, Product, Sales, Support, DevRel, Engineering, Machine Readers), each with exactly 1 concrete Cluvr project example:
 - Marketing → SEO-optimized clubs-directory copy
 - Product → apply-flow quick-start guide
 - Sales → feature doc aligned with CampusLoop pitch deck
 - Support → FAQ reducing "Apply button does nothing" tickets
 - DevRel → API quick-start for partner-club integrations
 - Engineering → hamburger-menu debug guide
 - Machine Readers → structured markdown as LLM/chatbot source

Scenario 2: Writer-in-the-Middle Role

- **GIVEN** the response must show understanding of my cross-team position (Chinchilla, p. 5)
- **WHEN** I frame my role in the opening of the response
- **THEN** I explicitly name the "writer-in-the-middle" concept and explain that I sit between engineering, product, marketing, sales, support, and DevRel to close information gaps and break down content silos — not just produce isolated documents

Scenario 3: End-End Users in Persona

- **GIVEN** the persona definition must go beyond the direct/primary user (Chinchilla, p. 7)
- **WHEN** I close the response by defining who benefits from my documentation
- **THEN** I explicitly include "end-end users" students who never touch the Cluvr platform directly but experience its output at events as part of who my documentation ultimately serves, and use this to justify allocating writing time in sprint planning

Acceptance Criteria Checklist (Self-Audit)

Шаардлага	Хангасан эсэх	Тайлбар
7 impact sphere, тус бүрт 1		Marketing (SEO directory), Product (apply-flow quick-start), Sales (feature doc + pitch deck), Support (FAQ), DevRel (API quick-start), Engineering (hamburger-menu debug guide), Machine Readers (structured markdown)

project example		
"Writer-in-the-Middle" ойлголт (p.5)	✓	Эхний догол мөрөнд шууд citation-той тодорхойлсон, багуудын хоорондох "content silos"-ыг холбогч гэдгийг тайлбарласан
"End-End Users" persona-д оруулсан (p.7)	✓	Сүүлийн догол мөрөнд — платформыг шууд ашигладаггүй, event дээр л туршлага хүртдэг оюутнуудыг тусад нь тодорхойлсон

Daily Standup

1. Which 3 open-source repos did I categorize into documentation types today?

Өнөөдөр Stripe, FastAPI, NumPy гэсэн 3 repository-г судалж, Getting Started, Tutorial, Reference, API Docs гэсэн 4 төрлөөр нь ангилсан.

2. Persona Card-ийн одоогийн явц ямар байгаа вэ?

Миний Persona Card болох Төгөлдөр-ийн мэдээллийг боловсруулж дуусгасан. Түүний technical role, өдөр тутам ашигладаг VS Code, Tailwind CSS, Chrome DevTools, GitHub Desktop, Figma зэрэг toolstack болон 3 гол pain point-ийг тодорхойлсон. Мөн хэрэглэгчийн байр сууринаас бичсэн quote-ийг оруулсан.

3. README-ээс ямар jargon өгүүлбэрийг өнөөдөр өөрчилсөн бэ?

Mobile Menu хэсгийн peer-* selector болон sibling-ийн тухай өгүүлбэрийг өөрчилсөн. Өмнө нь зөвхөн техникийн шалтгааныг тайлбарласан байсан бол одоо хамгийн түрүүнд юуг шалгах хэрэгтэйг хэлж, зөв болон буруу кодын жишээ нэмсэн.

Sprint Retrospective

1. Юу сайн болсон бэ?

Chinchilla-ийн 4 төрлийн documentation-ийг ашигласнаар documentation-ийг зүгээр хараад таамгаар биш, зориулалт болон хэрэглэгчийн хэрэгцээнд үндэслэн ангилж чадсан. Мөн Persona Card хийснээр README-г developer-ийн биш, хэрэглэгчийн өнцгөөс харахад тус болсон.

2. Юу цаг их авсан бэ?

README дээрх ямар өгүүлбэрийг jargon гэж үзэхээ тодорхойлоход нэлээд цаг зарцуулсан. Ялангуяа technical terms-ийг Persona-ийн мэдлэг, pain point-той харьцуулж байж аль хэсгийг өөрчлөх хэрэгтэйг шийдсэн.

3. Дараагийн долоо хоногт юуг өөрчлөх вэ?

Дараагийн долоо хоногт Persona болон documentation-оо эцэст нь шалгахын оронд ажлын явцад нь эрт validation хийхийг хичээнэ. Мөн хэрэглэгчид хамгийн хэрэгтэй мэдээллийг эхэнд нь өгөхөд илүү анхаарна.

Weekly Reflection 1

Bhatti et al.-ийн ямар ойлголт өмнөх documentation бичих дадлыг чинь хамгийн их өөрчилсөн бэ?

Надад хамгийн их нөлөөлсөн ойлголт нь “Curse of Knowledge”, Chapter 1, p.3 байсан. Өмнө нь би peer-*, PostCSS pipeline, Oxc/SWC зэрэг technical terms-ийг бусад developer-ууд ойлгоно гэж боддог байсан. Харин Persona хийх үед зарим хэрэглэгч эдгээр нэр томъёог мэдэхгүй байж болохыг ойлгосон. Тиймээс documentation бичихдээ өөрийн мэдлэг дээр биш, уншигчийн мэдлэгийн түвшин дээр анхаарах хэрэгтэй гэдгийг ойлгосон.

Weekly Reflection 2

Chinchilla-ийн ямар framework эсвэл дүрмийг цаашдын professional project-д ашиглах вэ?

Би Chinchilla-ийн Chapter 2, pp.9–24 дээрх documentation-ийн 4 үндсэн төрлийг цаашид ашиглана. Үүнд Getting Started, Tutorial, Reference, API Docs орно. Энэ нь хэрэглэгч ямар мэдээлэл хайж байгаагаас хамаарч зөв төрлийн documentation бэлтгэхэд тусална. Ирээдүйн software project дээр бүх мэдээллийг нэг README-д хийхийн оронд зориулалтаар нь салгаж бичихэд хэрэгтэй гэж бодож байна.

Weekly Reflection 3

Textbook-ийн documentation стандарт болон software team-ийн бодит ажиллагааны хооронд хамгийн том ялгаа хаана байна вэ?

Миний бодлоор хамгийн том ялгаа нь textbook хэрэглэгчийн хэрэгцээ, ойлголтын түвшинд тохируулж бичихийг шаарддаг боловч бодит software team-д documentation-ийг ихэвчлэн developer-ууд өөрсдийн ойлголтын түвшинд бичдэг явдал юм. Жишээлбэл, Cluvr-ийн

README-д peer-* selector, sibling, PostCSS pipeline зэрэг technical terms байсан ч хэрэглэгч яг юу хийх хэрэгтэйг тодорхой заагаагүй байсан. Үүнийг засахдаа хамгийн чухал мэдээллийг эхэнд өгч, дараа нь code example болон technical explanation нэмсэн.